



TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP
KHOA: KHOA HỌC ỨNG DỤNG.

CHƯƠNG 2

PHÂN TÍCH TẬP TIN XML, JSON

Giảng viên: TS.Cao Diệp Thắng
Khoa: Khoa học Ứng dụng

Chương 2.

MỤC TIÊU

- + Cách định dạng và cấu trúc dữ liệu của tập tin XML và JSON.
- + Các thư viện tích hợp sẵn để phân tích dữ liệu XML và JSON.
- + Cách xử lý các trường hợp lỗi có thể xảy ra, như thiếu thuộc tính, phần tử hoặc cú pháp không đúng.
- + Cách gửi và nhận dữ liệu qua API: Khi làm việc với tập tin XML và JSON từ nguồn trực tuyến

NỘI DUNG BÀI HỌC

2.1

TÀI LIỆU XML

2.2

PYTHON JSON

MỤC TIÊU BÀI HỌC

Sau khi học xong bài này, người học sẽ nắm được các vấn đề sau:

- Cách định dạng và cấu trúc dữ liệu của tập tin XML và JSON.
- Các thư viện tích hợp sẵn để phân tích dữ liệu XML và JSON.
- Cách xử lý các trường hợp lỗi có thể xảy ra, như thiếu thuộc tính, phần tử hoặc cú pháp không đúng.
- Cách gửi và nhận dữ liệu qua API: Khi làm việc với tập tin XML và JSON từ nguồn trực tuyến

2.2. PYTHON JSON

2.2.1 Khái niệm và cấu trúc JSON.

Thông thường khi làm các ứng dụng web, dữ liệu được lưu trong một trong các hệ quản trị cơ sở dữ liệu như **MySQL, SQL Server**... Tuy nhiên, nếu sử dụng những hệ thống này, việc cài đặt có thể phức tạp và gây phiền hà, đặc biệt với các ứng dụng nhỏ, gọn, đơn giản. Trước đây, lập trình viên thường chọn **SQLite** hoặc **XML** để lưu trữ, nhưng hiện nay, *JSON trở thành một lựa chọn phổ biến* nhờ tính gọn nhẹ và hỗ trợ rộng rãi theo chuẩn **RFC 8259**. Khái niệm JSON là gì sẽ được chúng ta tìm hiểu trong các nội dung sau.

2.2.1 Khái niệm và cấu trúc JSON.

a. JSON là gì?

- JSON (**J**ava**S**cript **O**bject **N**otation) là **định dạng văn bản** dùng để biểu diễn dữ liệu có cấu trúc, nhẹ và dễ đọc/ghi.
- JSON tổ chức dữ liệu bằng cặp **khóa–giá trị** trong đối tượng (object) và *danhsách có thứ tự* trong mảng (array).
- JSON độc lập với ngôn ngữ và được hầu hết ngôn ngữ lập trình hỗ trợ.
- Trong Python, ta dùng mô-đun chuẩn json để phân tích cú pháp (*parse*) và tuần tự hoá (*serialize*).

b. Dùng JSON khi nào?

- Trao đổi dữ liệu giữa client–server (REST API, Web API).
- Lưu cấu hình, dữ liệu nhỏ/gọn, hoặc xuất/nhập dữ liệu (import/export).
- Ghi log theo dòng (NDJSON) cho xử lý phân tích dữ liệu.



Khi cần truy vấn phức tạp, ràng buộc toàn vẹn, đồng thời truy cập cao, hãy dùng hệ quản trị CSDL (như SQLite/MySQL/PostgreSQL).

JSON có thể lưu trong CSDL (cột kiểu JSON) nhưng không thay thế CSDL.

Ví dụ 2.2.1.1a. Minh họa chuỗi JSON lưu trữ thông tin giảng viên, sinh viên

```
{
  "lop": "K18A",
  "mon_hoc": "Python nâng cao",
  "sinh_vien": [
    { "ho_ten": "Nguyen Van A", "nam_sinh": 2004, "diem_tb": 8.2 },
    { "ho_ten": "Tran Van B", "nam_sinh": 2003, "diem_tb": 7.6 }
  ],
  "giang_vien": { "ten": "CT", "email": "ct@uneti.edu" },
  "dang_mo": true,
  "ghi_chu": null
}
```

- Khối bao ngoài { ... } là đối tượng (object): gồm cặp **khóa–giá trị** như "lop": "K18A", "dang_mo": true.
- "sinh_vien": [...] là mảng (array) có thứ tự gồm 3 phần tử; mỗi phần tử lại là đối tượng sinh viên.
- "dang_mo": true → kiểu boolean (JSON viết lowercase true/false).
- "ghi_chu": null → giá trị null.

Ví dụ 2.2.1.1a. Dữ liệu chuỗi JSON lưu trữ thông tin giảng viên, sinh viên trong một tệp (file vd2.1.1a.json)

```
1  {
2    "lop": "K18A",
3    "mon_hoc": "Python nâng cao",
4    "sinh_vien": [
5      { "ho_ten": "Nguyen Van A", "nam_sinh": 2004, "diem_tb": 8.2 },
6      { "ho_ten": "Tran Van B",    "nam_sinh": 2003, "diem_tb": 7.6 }
7    ],
8    "giang_vien": { "ten": "CT", "email": "ct@uneti.edu" },
9    "dang_mo": true,
10   "ghi_chu": null
11 }
```


c. Cú pháp & kiểu dữ liệu cốt lõi

- Khóa và chuỗi phải dùng nháy kép "\"" (không dùng "nháy cong").
- Kiểu hợp lệ: *string*, *number*, *object*, *array*, *true*, *false*, *null*.
- Không có comment; không dấu phẩy thừa cuối phần tử.

d. Một số qui định:

❑ JSON có thể biểu diễn đối tượng { ... } hoặc mảng [...].

Thực tế khi lưu file .json, thường chọn đối tượng hoặc mảng làm gốc để xử lý.

❑ Khóa (**key**) và giá trị (**value**) :

- Khóa (key) của JSON bắt buộc phải là chuỗi đặt trong dấu nháy kép "\".

Các key của JSON nên đặt chữ cái không dấu hoặc số, dấu _ và không có khoảng trắng., ký tự đầu tiên không nên đặt là số.

❑ Giá trị (value) có thể là:

- **string** (nháy kép), **number** (không nháy), **object** {}, **array** [], **true**, **false**, **null**.

❑ Ký tự thoát (escaping) trong chuỗi:

- Hiển thị chính dấu \'\' bên trong chuỗi JSON, người dùng phải viết \'\\\' trong file văn bản gốc của JSON (hay nội dung gõ vào file .JSON hoặc chuỗi JSON trước khi parse)

d. Một số qui định:

❑ JSON có thể biểu diễn đối tượng { ... } hoặc mảng [...].

Thực tế khi lưu file .json, thường chọn đối tượng hoặc mảng làm gốc để dễ xử lý.

❑ Khóa (**key**) và giá trị (**value**) :

- Khóa (key) của JSON bắt buộc phải là chuỗi đặt trong dấu nháy kép "...".

Các key của JSON nên đặt chữ cái không dấu hoặc số, dấu _ và không có khoảng trắng., ký tự đầu tiên không nên đặt là số.

❑ Giá trị (value) có thể là:

- **string** (nháy kép), **number** (không nháy), **object** {}, **array** [], **true**, **false**, **null**.

❑ Ký tự thoát (escaping) trong chuỗi:

- Hiển thị chính dấu \'\' bên trong chuỗi JSON, người dùng phải viết \'\' trong file văn bản gốc của JSON (hay nội dung gõ vào file .JSON hoặc chuỗi JSON trước khi parse)

Lưu ý: *File JSON có thể được lưu với bất kỳ phần mở rộng nào, tuy nhiên dạng thường được sử dụng là: **.json** hoặc **.js***

Minh họa : biểu diễn ký tự thoát (escaping) trong chuỗi JSON

Nguồn JSON (chuỗi)	Giá trị sau khi phân tích cú pháp (parse)
"A\\B"	A\B
"C:\\"	C:\
"Nói \\\"JSON\\\""	Nói "JSON"
{ "path": "C:\\Users\\CT\\Documents" }	path = C:\Users\CT\Documents

e. Bẫy thường gặp

1. Nháy đơn/nháy cong:

Sai: { 'x': 1 } hoặc { "x": 1 }

2. Dấu phẩy thừa:

Sai: {"x":1,} [1, 2,]

3. Comment trong JSON

Sai: {

 //Ghi chú

 "x": 1

}

4. Thoát ký tự không chuẩn trong chuỗi

Sai: { "note": "Nói "JSON" là gì?" }

5. Dùng True/False/None của Python trong file JSON

Sai: { "ok": True, "desc": None }

Đúng: { "x": 1 }

JSON bắt buộc dùng nháy kép (") cho khóa và chuỗi.

Đúng: {"x":1}[1, 2]

Đúng: Không có comment.

Nếu cần chú thích, thêm trường dữ liệu "note" riêng.

Đúng: { "note": "Nói \\\"JSON\\\" là gì?" }

Dùng \" cho dấu ", \\ cho backslash, \n xuống dòng, v.v.

Đúng: { "ok": true, "desc": null }

Khi parse vào Python:

true → **True**, **false** → **False**, **null** → **None**.

e. Bẫy thường gặp

- **Khóa trùng** bị ghi đè: { "x":1, "x":2 } → kết quả thường là { "x":2 }.
- **Số không chuẩn**: 01, +1, NaN, Infinity → dùng 0, 1, -1, 3.14.
- **Chuỗi JSON nối liền** không có phân tách: { "a":1 } { "b":2 } → dùng mảng [{...},{...}] hoặc **NDJSON** (mỗi dòng một JSON).
- **Mã hóa**: lưu/đọc **UTF-8**; tránh BOM.
- **Kiểu không thuộc JSON** (datetime/Decimal/Set) → chuyển **chuỗi ISO**/số epoch hoặc quy ước encoder/decoder

Ưu và nhược điểm của JSON



Ưu điểm:

- **Định dạng đơn giản:** JSON được thiết kế để đọc và viết một cách dễ dàng cho con người, đây là một định dạng dữ liệu đơn giản, dễ hiểu và đọc.
- **Hỗ trợ nhiều ngôn ngữ lập trình:** JSON được hỗ trợ bởi hầu hết các ngôn ngữ lập trình phổ biến như JavaScript, Python, Ruby, PHP và nhiều ngôn ngữ khác.
- **Kích thước nhỏ:** JSON có kích thước nhỏ hơn so với một số định dạng dữ liệu khác, giúp cho việc truyền tải và lưu trữ dữ liệu dễ dàng hơn.
- **Dữ liệu có cấu trúc:** JSON sử dụng cấu trúc dữ liệu **key-value**, mảng và đối tượng, cho phép xử lý dữ liệu phức tạp hơn.
- **Hỗ trợ tương tác với các API:** JSON được sử dụng rộng rãi để truyền dữ liệu giữa các ứng dụng và tương tác với các API (Application Programming Interface).

Nhược điểm

- Không có hỗ trợ cho các tính năng chuyên biệt: JSON không cung cấp các tính năng chuyên biệt như kiểm tra lỗi, bảo mật dữ liệu, nén dữ liệu, v.v.
- Không hỗ trợ các kiểu dữ liệu phức tạp: JSON không hỗ trợ các kiểu dữ liệu phức tạp như ngày giờ, số thập phân, v.v. nên trong một số trường hợp, nó có thể gây khó khăn trong việc lưu trữ và truyền tải dữ liệu.
- Dữ liệu chỉ có thể được sử dụng khi được giải mã: Dữ liệu JSON cần được giải mã bởi ngôn ngữ lập trình trước khi có thể sử dụng. Việc này có thể tốn nhiều thời gian và tài nguyên hệ thống nếu dữ liệu lớn hoặc phức tạp.

Một số ví dụ chuỗi JSON.

Ví dụ 2.2.1.2:

Có 03 sinh viên với các thông tin như sau: Nguyễn Văn A – sinh năm 2004, Trần Văn C – sinh năm 2003, Lê Thị B – sinh năm 2004. Hãy xây dựng chuỗi JSON lưu trữ thông tin danh sách các sinh viên nói trên.

1	{
2	"sinhvien1_ten" : "Nguyễn Văn A",
3	"sinhvien1_nam_sinh" : 2003,
4	"sinhvien2_ten" : "Trần Văn C",
5	"sinhvien2_nam_sinh" : 2003,
6	"sinhvien3_ten" : "Lê Thị B",
7	"sinhvien3_nam_sinh" : 2004
8	}

➡ Quan sát chuỗi JSON trên chúng ta thấy khá bất tiện, do thông thường mỗi đối tượng sẽ được gộp vào một nơi để thuận tiện cho việc lưu trữ, tìm kiếm và phân chia,... 15

Xây dựng cách khác cho chuỗi JSON trên.

Ví dụ 2.2.1.3.

Sử dụng
dấu '['
' để
gom nhóm
lại.

```
1  [
2      {
3          "ho_va_ten" : "Nguyễn Văn A",
4          "nam_sinh" : 2004
5      },
6      {
7          "ho_va_ten" : "Nguyễn Văn C",
8          "nam_sinh" : 2003
9      },
10     {
11         "ho_va_ten" : "Lê Thị B",
12         "nam_sinh" : 2004
13     }
14 ]
15
```

Sử dụng được cách nhau bởi dấu phẩy (,).

Ví dụ 2.2.2.1

```
1  '{
2      "firstName": "A",
3      "lastName": "Nguyễn Văn",
4      "hobbies": ["Đá banh", "Lái xe", "Chụp ảnh"],
5      "age": 35,
6      "children": [
7          {
8              "firstName": "Minh",
9              "age": 7
10         },
11         {
12             "firstName": "Anh",
13             "age": 4
14         }
15     ]
16 }'
```

Ví dụ 2.2.1.4. Một trường đại học lưu trữ điểm của sinh viên, mỗi sinh viên sẽ có một mã số sinh viên sẽ đăng ký học theo các tín chỉ khác nhau, hãy tổ chức cấu trúc chuỗi JSON để lưu trữ danh sách sinh viên và danh sách các môn học của sinh viên đó.

Biết rằng:

Sinh viên Nguyễn Văn A có mã số sinh viên là khud0001.

Sinh viên Nguyễn Văn C có mã số sinh viên là khud0005.

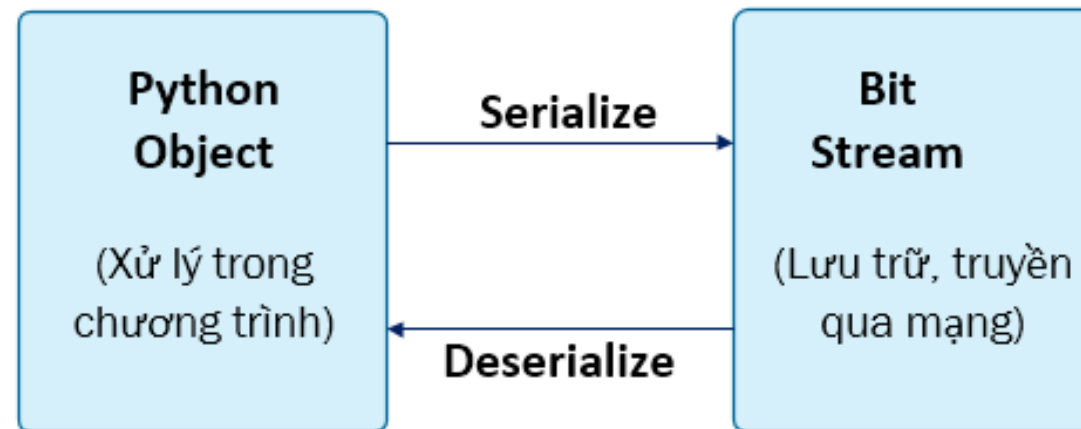
```
1  {  
2      "khud0001" : {  
3          "Lap_trinh" : "Python nâng cao",  
4          "Toán" : "Xác suất thống kê"  
5      },  
6      "khud0005" : {  
7          "Lap_trinh" : "Python nâng cao",  
8          "Ngoai_ngu" : "Anh văn 2"  
9      }  
10 }
```

2.2.2. Chuyển đổi giữa chuỗi Python <--> JSON

Giới thiệu

- **Serialize** (tuần tự hóa): Serialize là quá trình chuyển đổi đối tượng Python (như **dict**, **list**, **str**, **int**, **float**, **bool**, **None**) thành định dạng JSON (chuỗi hoặc file) để **lưu trữ** hoặc **truyền tải**.
- **Deserialize** (giải tuần tự hóa): Chuyển đổi ngược lại, tức là *đọc chuỗi JSON* hoặc *tệp JSON* và biến chúng thành các đối tượng Python để *xử lý trong chương trình*.

Python cung cấp sẵn **module json**, hỗ trợ đầy đủ cả hai quá trình trên, đảm bảo tương thích với chuẩn RFC 8259.



Kiểm tra xem module 'json' đã được cài đặt chưa? `import json`

Lưu ý: module **'json'** đã được tích hợp sẵn trong Python 3x, vì vậy người dùng không cần phải cài đặt nó một cách riêng lẻ.

2.2.2.1. Serialize (tuần tự hóa):

Bảng 2.2.a. Mô tả dữ liệu được chuyển đổi giữa Python và JSON

Python	JSON
dict	object
list, tuple	array
str	string
int, long, float	number
Bool (True/False)	True/false
None	null

Lưu ý:

- ❑ **tuple** khi serialize dùng hàm **dump** → **array**; khi deserialize dùng hàm **load** lại, array → **list**
- ❑ **int** của Python 3 có độ chính xác tùy ý; nếu gửi sang môi trường JSON có thể vượt quá 53-bit, cần nhắc lưu dạng chuỗi.
- ❑ Các kiểu không được hỗ trợ mặc định (*bytes, set, Decimal, datetime, ...*) cần tự mã hoá qua `default=` hoặc chuyển sang chuỗi/số phù hợp.

a. Serialize,... tiếp

Sử dụng công cụ chính:

- `json.dumps()`: Chuyển đối tượng Python thành chuỗi JSON.
- `json.dump()`: Chuyển đổi đối tượng Python thành chuỗi JSON và ghi tiếp vào file JSON.

Cú pháp hàm `json.dumps()`

```
json_string = json.dumps(obj, *, skipkeys=False, ensure_ascii=True, check_circular=True,  
                           allow_nan=True, cls=None, indent=None, separators=None,  
                           default=None, sort_keys=False, **kw)
```

Một số đối số chính :

obj: (Bắt buộc) Đối tượng Python muốn chuyển đổi thành chuỗi JSON.

ensure_ascii: Đặt False để giữ nguyên ký tự Unicode (mặc định True chuyển thành escape sequence).

VD: `json.dumps({"name": "Hiệu"}, ensure_ascii=False) → {"name": "Hiệu"}`

indent: (Tùy chọn) mặc định là none thêm định dạng dễ đọc với số khoảng trắng (ví dụ: indent=2)

Phương thức **dumps()**, ...



Default: mặc định là None. Cho phép xác định một hàm hoặc phương thức để xử lý các đối tượng không thể mã hóa JSON một cách tùy chỉnh.

```
import json

def complex_handler(obj):
    if isinstance(obj, complex):
        return [obj.real, obj.imag]
    raise TypeError("Unknown type")

data = {"num": 3 + 4j}
json_str = json.dumps(data, default=complex_handler)
print(json_str)  # {"num": [3.0, 4.0]}
```

sort_keys: (Tùy chọn) Sắp xếp key theo thứ tự alphabet (mặc định là False.)

`json.dumps({"z": 1, "a": 2}, sort_keys=True) → {"a": 2, "z": 1}.`

****kw** : cho phép truyền một số tham số tùy chỉnh khác cho quá trình mã hóa JSON. Tham số ****kw** là một cách để truyền các tham số dưới dạng từ điển (dictionary) với tên tham số là các khóa và giá trị tham số là các giá trị tương ứng.



Lưu ý: Có thể tìm hiểu thêm về các tham số và chức năng của hàm `json.dumps()` trong tài liệu chính thức của Python [].

Ví dụ 2.2.2.1.1 Giả sử chúng ta có đối tượng Python Object là:

`data = {"name": "A", "age": 35, "company": "Đất Việt"},`

hãy chuyển đổi đối tượng python object, data ở trên thành một chuỗi JSON (Json string)

```
1 import json
2
3 # Đối tượng Python (đối tượng dict)
4 data = {
5     "Ho_va_ten": "Ng Văn A",
6     "Nam_sinh": 2025,
7     "Gioi_tinh": "Nam",
8     "so_dt": "0123456789",
9     "email": "nguyenvana@uneti.edu.vn"
10 }
11
12 # Chuyển đổi đối tượng Python thành chuỗi JSON
13 json_string = json.dumps(data, ensure_ascii=False)
14
15 print(json_string)
16 print(type(json_string))
```

đặt tham số `ensure_ascii=False` để hiển thị các ký tự Unicode dưới dạng nguyên thủy, Trường hợp ngược lại, chuỗi 'Đất Việt' sẽ hiển thị ở dạng:

`"\u0110\u1ea5t Vi\u1ec7t"`

Nguyên nhân là do việc chuyển đổi Unicode trong chuỗi thành escape sequences.

Kết quả thực hiện chương trình:

```
{"name": "A", "age": 35, "company": "\u0110\u1ea5t Vi\u1ec7t"}
<class 'str'>
```

❑ Hàm **dump()**. chuyển đổi một đối tượng Python (ví dụ: dictionary, list) thành chuỗi JSON và ghi trực tiếp chuỗi JSON đó vào một tệp (file) hoặc đối tượng file-like.

Ví dụ 2.2.2.1.2

```
1     import json
2
3     # Đối tượng Python (đối tượng dict)
4     data = {
5         "Ho_va_ten": "Nguyễn Văn A",
6         "Nam_sinh": 2025,
7         "Gioi_tinh": "Nam",
8         "so_dt": "0123456789",
9         "email": "nguyenvana@uneti.edu.vn"
10    }
11
12
13
14    # Sử dụng 'with open' để mở tệp 'thongtin.json' với chế độ ghi ('w') và mã hóa utf-8
15    # Hàm json.dump() sẽ ghi đối tượng Python trực tiếp vào tệp đã mở.
16    with open('thongtin.json', 'w', encoding='utf-8') as f:
17        json.dump(data, f, ensure_ascii=False, indent=4) #Hàm dump() ghi đối tượng Python
18        vào file thongtin.json
19
20
21    print("Dữ liệu đã được ghi thành công vào tệp 'thongtin.json'.")
```

Kết quả thực hiện chương trình: Dữ liệu đã được ghi thành công vào tệp 'thongtin.json'.

Lưu ý: sự khác biệt chính giữa `json.dump()` và `json.dumps()` (vì `json.dumps()` chỉ trả về chuỗi JSON, không ghi vào file).

Xử lý những object mà module json không hỗ trợ

- ❖ Module json hỗ trợ serialize/deserialize hầu hết các kiểu dữ liệu được xây dựng sẵn (built-in types), tuy nhiên có một vài kiểu dữ liệu mà module này không hỗ trợ.

Ví dụ :

- Số phức (complex) không được module Json hỗ trợ khi serialize/deserialize.
 - Các kiểu dữ liệu do người dùng tự định nghĩa (user-defined types), vì nó không có thông tin gì về các kiểu dữ liệu này. Nếu chúng ta xử lý như bình thường, sẽ có lỗi xảy ra.
-
- ❖ Để serialize/deserialize các kiểu dữ liệu này, chúng ta cần cung cấp thêm cho Module Json các trợ giúp: **default** cho serialize và **object_hook** cho deserialize.

▪ Mã hóa đối tượng phức trong python

Một đối tượng phức có hai phần khác nhau đó là phần thực, phần ảo.

$$\begin{array}{cc} a & + & bi \\ \uparrow & & \uparrow \\ \text{phần thực} & & \text{phần ảo} \end{array}$$

Ví dụ 2.2.2.1.3. Mã hóa 5+3i

```
In[ ] 1 import json
      2
      3 # Viết một hàm để kiểm tra xem một đối tượng có phải là số phức hay không.
      4 def complex_encode(object):
      5     # Kiểm tra sử dụng isinstance method
      6     if isinstance(object, complex):
      7         return [object.real, object.imag]
      8     # Kích hoạt lỗi sử dụng exception handling nếu object không phải là số phức
      9     raise TypeError(repr(object) + " is not JSON serialized")
     10
     11 data = 5+3j #Data là đối tượng số phức
     12 # Sử dụng hàm json.dumps() để mã hóa số phức 5 + 3j thành chuỗi JSON.
     13 complex_obj = json.dumps(data, default=complex_encode)
     14
     15 print(complex_obj)
```

➡ [5.0, 3.0]

Ví dụ 2.2.2.1.4.a

```
In[ ] 1 import json
      2
      3 # Viết một hàm để kiểm tra xem một đối tượng có phải là số phức hay không.
      4 def complex_encode(object):
      5     # Kiểm tra sử dụng isinstance method
      6     if isinstance(object, complex):
      7         return [object.real, object.imag]
      8     # Kích hoạt lỗi sử dụng exception handling nếu object không phải là số phức
      9     raise TypeError(repr(object) + " is not JSON serialized")
     10
     11 data = {1,2,3}          #Data không là đối tượng số phức
     12 # Sử dụng hàm json.dumps() để mã hóa số phức 4 + 3j thành chuỗi JSON.
     13 complex_obj = json.dumps(data, default=complex_encode)
     14
     15 print(complex_obj)
```

Out[:] **TypeError:** {1, 2, 3}is Not JSON serialized

2.2.2.2. DeSerialize (JSON → Python):

Sử dụng công cụ chính:

- `json.loads()`: Chuyển thành chuỗi JSON thành đối tượng Python .
- `json.load()`: Chuyển đổi chuỗi JSON từ file JSON thành đối tượng Python

Cú pháp: `json.loads(json_string, *, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw)`

Trong đó:

- **json_string**: Đây là chuỗi JSON chúng ta muốn chuyển đổi thành các đối tượng Python.
- **object_hook**: (Tùy chọn) Đây là một hàm tùy chọn được sử dụng để xử lý các đối tượng JSON. Nó được gọi cho mỗi đối tượng trong JSON và cho phép thực hiện các tùy chỉnh hoặc biến đổi khi chuyển đổi JSON thành các đối tượng Python.
- **parse_float, parse_int, parse_constant**: (Tùy chọn) Các tham số này là các hàm tùy chọn được sử dụng để chuyển đổi số dấu phẩy động, số nguyên và các hằng số JSON (ví dụ: NaN, Infinity) thành các kiểu số Python tương ứng.
- **object_pairs_hook**: (Tùy chọn) Đây là một hàm tùy chọn được sử dụng để xử lý các cặp khóa-giá trị JSON khi chuyển đổi JSON thành các đối tượng Python.

- Giải mã đối tượng JSON phức trong Python

Sử dụng tham số **object_hook** để kiểm tra chuỗi JSON có chứa đối tượng phức.

Ví dụ 2.2.2.2.1. Sử dụng tham số **object_hook** trong hàm **json.loads()** để kiểm tra và chuyển đổi các đối tượng đặc biệt (complex objects) khi phân tích một chuỗi JSON thành Python.

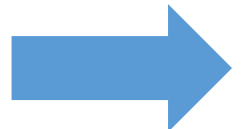
Giả sử có chuỗi JSON **complex_object** với nội dung:

```
'{"__complex__":true,"real":4,"img":5}'
```

Và chuỗi JSON **simplex_object** có nội dung:

```
'{"real":6,"img":7}'
```

Chúng ta viết code 2.2.2.5.ipynb như sau:



Ví dụ 2.2.2.2.1. (tiếp, ...)

```
1 import json
2
3 # Hàm kiểm tra chuỗi JSON có chứa đối tượng phức (complex object) hay không
4 def is_complex(object):
5     # Kiểm tra xem trong đối tượng có tồn tại khóa '__complex__' hay không
6     if '__complex__' in object:
7         # Nếu có, tạo một số phức từ thông tin trong đối tượng
8         return complex(object['real'], object['img'])
9     # Nếu không có khóa '__complex__', đối tượng không phải là đối tượng phức,
10    # trả về đối tượng gốc
11    return object
12
13 def main():
14     # Sử dụng hàm json.loads() với object_hook để kiểm tra và chuyển đổi đối tượng
15     # phức trong chuỗi JSON
16     complex_object = json.loads('{"__complex__": true, "real": 4, "img": 5}',
17                                object_hook=is_complex)
18     simple_object = json.loads('{"real": 6, "img": 7}',
19                                object_hook=is_complex)
20
21     # In kết quả phân tích chuỗi JSON cho cả đối tượng phức và đối tượng thường
22     print("Đối tượng số phức.....", complex_object)
23     print("Đối tượng bình thường.....", simple_object)
24
25 if __name__ == "__main__":
26     main()
```

b. Chuyển chuỗi JSON thành đối tượng Python (Python Object)

Thực chất là chuyển **một chuỗi string có cấu trúc JSON** thành một **object** trong Python.

Lưu ý:

- Python cho phép sử dụng cả nháy đơn và nháy kép để biểu thị một chuỗi JSON.
- Bất kỳ chuỗi nào được bao quanh bởi nháy đơn hoặc nháy kép đều được coi là chuỗi hợp lệ.

Trường hợp muốn bao quanh bằng chuỗi nháy kép:

Kỹ thuật "escaping" (tránh xung đột ký tự).

Ví dụ: xét một cấu trúc chuỗi JSON như sau:

```
'{"name": "A", "age": 35, "company": "Đất Việt"}'
```

Nếu muốn biểu diễn chuỗi Json trên bao quanh bằng cặp nháy kép chúng ta cần viết:

```
"{\"name\": \"A\", \"age\": 35, \"company\": \"Đất Việt\"}"
```

d. Chuyển chuỗi JSON thành đối tượng Python

❑ **Phương thức `json.loads()`** chuyển đổi chuỗi JSON thành các đối tượng Python với nhiều tùy chỉnh, giúp linh hoạt hóa quá trình xử lý dữ liệu JSON phức tạp.

Cú pháp `data = json.loads(s, *, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None)`

- **s** (bắt buộc): Đây là chuỗi JSON mà ta muốn chuyển đổi thành đối tượng Python. Tham số này là cần thiết.
- *****: Dấu * trong cú pháp hàm là một cách ép Python chỉ cho phép các tham số sau nó được truyền vào dưới dạng tham số từ khóa (keyword arguments). Nghĩa là cần phải chỉ rõ tên của các tham số từ `cls` trở đi khi gọi hàm, như `cls=...`, `object_hook=...`
- **cls** (tùy chọn): Cho phép sử dụng một lớp tùy chỉnh để giải mã dữ liệu JSON, thay thế lớp mặc định `json.JSONDecoder`. Điều này hữu ích khi chúng ta muốn tùy chỉnh quá trình giải mã theo cách riêng.
- **object_hook** (tùy chọn): Là một hàm cho phép tùy chỉnh cách mà các đối tượng JSON được giải mã. Khi `json.loads()` gặp một đối tượng JSON, nó sẽ truyền đối tượng này vào hàm `object_hook`, và hàm này sẽ trả về kết quả mong muốn (có thể là một kiểu đối tượng khác hoặc cấu trúc tùy chỉnh). Ví dụ, có thể sử dụng `object_hook` để chuyển đổi đối tượng JSON thành một `namedtuple` hoặc một lớp tùy chỉnh thay vì dictionary.

❑ Phương thức `json.loads()`, tiếp ...

- **`parse_int`** (tùy chọn): Là một hàm cho phép tùy chỉnh cách giải mã các số nguyên (integer) trong JSON. Mặc định, JSON sẽ chuyển các số nguyên thành kiểu `int` của Python, nhưng ta có thể chỉ định hàm này để chuyển thành kiểu khác, chẳng hạn như `float` hoặc `str`.
- **`parse_constant`** (tùy chọn): Là một hàm dùng để xử lý các giá trị đặc biệt trong JSON như `"NaN"`, `"Infinity"`, và `"-Infinity"`. Nếu các giá trị này xuất hiện trong chuỗi JSON và không có hàm nào được chỉ định trong `parse_constant`, Python sẽ tạo ra lỗi `ValueError`. Chúng ta có thể dùng `parse_constant` để tùy chỉnh xử lý các giá trị đặc biệt này.
- **`object_pairs_hook`** (tùy chọn): Là một hàm cho phép xử lý các cặp khóa-giá trị trong đối tượng JSON theo một cách đặc biệt. Mặc định, JSON sẽ chuyển đối tượng thành dictionary (`dict`), nhưng với `object_pairs_hook`, ta có thể giữ thứ tự các cặp khóa-giá trị hoặc sử dụng một kiểu dữ liệu khác, chẳng hạn như `OrderedDict`, để giữ nguyên thứ tự của các khóa.



Lưu ý: Có thể tìm hiểu thêm về các tham số và chức năng của hàm `json.loads()` trong tài liệu chính thức của Python.

Ví dụ 2.2.2.2a.

```
1  import json
2  from decimal import Decimal      #cung cấp lớp Decimal để hỗ trợ cho các phép tính số học
3                                     #thập phân độ chính xác cao
4
5
6  json_string = '{"name": "Alice", "age": 25, "balance": 1000.5, "status": "NaN"}'
7
8  data = json.loads(
9      json_string,
10     parse_float=Decimal,          # Chuyển đổi float thành Decimal
11     parse_int=str,                # Chuyển đổi int thành chuỗi
12     parse_constant=lambda x: None # Chuyển các giá trị đặc biệt thành None
13 )
14
15 print(data)
```

Out:

```
{'name': 'Alice', 'age': '25', 'balance': Decimal('1000.5'), 'status': 'NaN'}
```

Ví dụ 2.2.2.2



In[]	<pre>1 import json 2 3 # Chuỗi JSON bao quanh bằng dấu nháy đơn 4 json_string_1 = '{"name": "A", "age": 35, "company": "Đất Việt"}' 5 6 # Chuỗi JSON bao quanh bằng dấu nháy kép 7 json_string_2 = "{\"name\": \"A\", \"age\": 35, \"company\": \"Đất Việt\"}" 8 9 # Chuyển chuỗi JSON thành đối tượng Python (đối tượng JSON) 10 json_object_1 = json.loads(json_string_1) 11 json_object_2 = json.loads(json_string_2) 12 13 print(json_object_1) 14 print(json_object_2)</pre>
Kết quả thực hiện chương trình:	
	<pre>{'name': 'A', 'age': 35, 'company': 'Đất Việt'} <class 'dict'></pre>

d. Chuyển file JSON thành đối tượng Python...

Hàm **load()** đọc nội dung của một file JSON thành một đối tượng trong Python

Ví dụ 2.2.2.2.3. Giả sử chúng ta có file **thongtin.json** có nội dung như sau:

```
{  
    "Ho_va_ten": "Nguyễn Văn A",  
    "Nam_sinh": 2025,  
    "Gioi_tinh": "Nam",  
    "so_dt": "0123456789",  
    "email": "nguyenvana@uneti.edu.vn"  
}
```

Yêu cầu: đọc nội dung file này và chuyển nội dung đó thành một đối tượng trong Python.

Ví dụ 2.2.2.2.3.

```
1 import json
2
3 with open('thongtin.json') as f:
4     infor = json.load(f)
5     print(infor)
6     print(type(infor))
```

Kết quả thực hiện chương trình:

```
{'ho_va_ten': 'Nguyễn Văn A', 'nam_sinh': 2004,
'gioi_tinh': 'Nam', 'sdt': '0123456789', 'email':
'nguyenvana@uneti.e56789', 'email':
'nguyenvana@uneti.edu.vn'}
<class 'dict'>
```

Nhận xét: trong kết quả trên ta thấy chuỗi "Nguyễn Văn A" khi chuyển sang đối tượng đã bị hiển thị sai do dữ liệu trong file JSON mã hóa utf-8. Do vậy có thể sửa lại code ở ví dụ 2.2.2.2.3 như sau:

Ví dụ 2.2.2.2.3a.

```
1 import json
2
3 # Đọc nội dung tệp với mã hóa UTF-8
4 with open('thongtin.json', 'r', encoding='utf-8') as f:
5     content = f.read()
6
7 # Phân tích nội dung JSON từ chuỗi UTF-8
8 infor = json.loads(content)
9
10 print(infor)
11 print(type(infor))
```

Kết quả thực hiện chương trình:

```
{'ho_va_ten': 'Nguyễn Văn A', 'nam_sinh': 2004, 'gioi_tinh':  
'Nam', 'sdt': '0123456789', 'email': 'nguyenvana@uneti.edu.vn'}  
<class 'dict'>
```

Lưu ý: File JSON phải chứa nội dung đã được format sang định JSON và file JSON phải ở trong cùng thư mục với file chương trình, nếu không chúng ta phải chỉ ra đường dẫn đến file JSON.

So sánh `json.load()` và `json.loads()`

Hàm `json.load()` và `json.loads()` trong Python đều dùng để chuyển đổi dữ liệu JSON thành đối tượng Python (như dictionary hoặc list), nhưng chúng có sự khác biệt trong cách lấy dữ liệu JSON đầu vào:

`json.load()` và `json.loads()`

Đặc điểm	<code>json.load()</code>	<code>json.loads()</code>
Đầu vào	Đối tượng file (đọc từ file JSON)	Chuỗi JSON
Sử dụng khi	Đọc dữ liệu JSON từ tệp (file)	Đọc dữ liệu JSON từ một chuỗi
Ví dụ	<code>json.load(file)</code>	<code>json.loads(json_string)</code>

2.2.3. Tổng quan về lớp mã hóa(Serialization) JSON

- ❑ **JSONEncoder** là một lớp trong thư viện json của Python, được sử dụng để mã hóa (serialize) các đối tượng Python thành chuỗi JSON.
- ❑ Lớp **JSONEncoder** thực hiện việc chuyển đổi các kiểu dữ liệu Python thành cấu trúc dữ liệu JSON tuân thủ chuẩn JSON.
- ❑ Khi muốn chuyển đổi một **đối tượng Python** thành **chuỗi JSON**, chúng ta có thể tạo một thể hiện của lớp **JSONEncoder** hoặc một *lớp con tùy chỉnh (kế thừa)* từ lớp JSONEncoder.

JSONEncoder có 3 phương thức thực hiện mã hóa:

- **default(obj)** - Được triển khai trong lớp con và trả về đối tượng tuần tự cho đối tượng obj.
- **encode(obj)** - Giống như phương thức json.dumps (), nó trả về chuỗi JSON của cấu trúc dữ liệu Python.
- **iterencode(obj)** – Biểu diễn từng chuỗi một và mã hóa đối tượng obj.

a. Phương thức mã hóa **default(self, obj)**

- Phương thức **default(self, obj)** là một phương thức trong lớp kế thừa từ **json.JSONEncoder** (hoặc lớp tùy chỉnh) được sử dụng để xác định cách chuyển đổi một đối tượng Python không thể **serialize** một cách mặc định thành một đối tượng có thể serialize và mã hóa thành chuỗi JSON.
- Khi **json.dumps()** gặp một đối tượng không thể **serialize**, nó sẽ gọi phương thức **default(self, obj)** của lớp **JSONEncoder** (hoặc lớp tùy chỉnh đã được người dùng định nghĩa) và truyền đối tượng không thể serialize đó làm tham số **obj**.
- Chúng ta có thể định nghĩa phương thức **default** để kiểm tra kiểu của đối tượng **obj** và trả về một đối tượng có thể serialize, mà sau đó sẽ được mã hóa thành JSON.

Phương thức **default(self, obj)** có thể trả về một trong các giá trị

- Đối tượng có thể serialize (thường là một từ điển, danh sách hoặc kiểu dữ liệu JSON hợp lệ khác).
- **None**: Nếu chúng ta muốn bỏ qua việc mã hóa đối tượng này và giữ nguyên nó.
- Raise một ngoại lệ (exception) như **TypeError** nếu chúng ta muốn từ chối việc mã hóa đối tượng và thông báo về lỗi.

Ví dụ 2.2.2.2.4a



```
1 import json
2 # Định nghĩa một lớp đối tượng tùy chỉnh
3 class CustomObject:
4     def __init__(self, data):
5         self.data = data
6     def to_dict(self):
7         return {"custom_data": self.data}
8
9 # Tạo một lớp JSONEncoder tùy chỉnh để xử lý đối tượng CustomObject
10 class CustomObjectEncoder(json.JSONEncoder):
11     def default(self, obj):
12         if isinstance(obj, CustomObject):
13             return obj.to_dict()
14             return None
15         return super().default(obj)
16
17 # Tạo một đối tượng CustomObject
18 custom_obj = CustomObject({'key': 'value'})
19
20 # Sử dụng JSONEncoder tùy chỉnh để mã hóa đối tượng CustomObject
21 json_data = json.dumps(custom_obj, cls=CustomObjectEncoder)
22
23 print(json_data)
```

➔ {"custom_data":
{"key": "value"}}

Ví dụ 2.2.2.2.5. Sử dụng lớp JSONEncoder từ module json.encoder để mã hóa một đối tượng Python thành một chuỗi JSON.



```
In[] 1 # Import Lớp JSONEncoder từ module json.encoder
      2 from json.encoder import JSONEncoder
      3
      4 # Định nghĩa từ điển colour_dict chứa một danh sách các màu.
      5 colour_dict = { "colour": ["red", "yellow", "green" ]}
      6
      7 # Sử dụng phương thức encode() của JSONEncoder để mã hóa từ điển colour_dict
      8 # thành chuỗi JSON.
      9 json_string=JSONEncoder().encode(colour_dict)
     10
     11 #in chuỗi json_string được mã hóa ra màn hình.
     12 print(json_string)
     13
     14 #in ra kiểu của chuỗi json_string
     15 print(type(json_string))
```

Kết quả thực hiện chương trình: {"colour": ["red", "yellow", "green"]}
<class 'str'>

2.2.4. Tổng quan về lớp giải mã (deserialization) JSON

Lớp **JSONDecoder** được sử dụng để giải mã tuần tự hóa bất kỳ đối tượng Python nào trong khi thực hiện giải mã. Nó chứa ba phương pháp giải mã khác nhau:

- **default (obj)** - Được triển khai trong lớp con và trả về đối tượng giải mã.
- **decode (obj)** - Giống như phương thức `json.loads()`, nó trả về cấu trúc dữ liệu Python của chuỗi hoặc dữ liệu JSON.
- **raw_decode (obj)** – Biểu diễn từng từ điển Python và giải mã đối tượng obj.

JSONDecoder là một lớp con của **json.JSONDecoder**, cho phép tùy chỉnh quá trình giải mã. Phương thức `decode()` sẽ giải mã chuỗi JSON và trả về đối tượng Python tương ứng.

Ví dụ 2.2.4.1.

```
colour_string = '{ "colour": ["red", "yellow"] }'
```

Kết quả thực hiện chương trình: `{'colour': ['red', 'yellow']}`
`<class 'dict'>`

2.2.5. Giải mã dữ liệu JSON từ URL.

Ví dụ thực tế: Chúng ta sẽ lấy dữ liệu của một số URL sau:
Dữ liệu của CityBike NYC (Hệ thống chia sẻ xe đạp) từ URL:

<https://feeds.citibikenyc.com/stations/stations.json>

JSONPlaceholder: JSONPlaceholder là một dịch vụ API giả lập để kiểm thử ứng dụng của người dùng với dữ liệu JSON. Dịch vụ này cung cấp các tài nguyên như bài đăng (posts), người dùng (users) và bình luận (comments) dưới dạng dữ liệu JSON, từ URL:

<https://jsonplaceholder.typicode.com/posts>

SpaceX API: SpaceX cung cấp API để truy xuất thông tin về các chuyến bay và tên lửa dưới dạng dữ liệu JSON. Chúng ta có thể sử dụng mã code trên để lấy thông tin từ API SpaceX. từ URL:

<https://api.spacexdata.com/v4/launches>

Lưu ý: Một số API có yêu cầu xác thực (API key) để sử dụng, ví dụ như OpenWeatherMap API. Trong trường hợp này, chúng ta cần đăng ký và nhận API key từ nhà cung cấp API trước khi sử dụng mã code. Hãy chắc chắn đọc tài liệu API để biết cách sử dụng và các yêu cầu cụ thể của từng API.

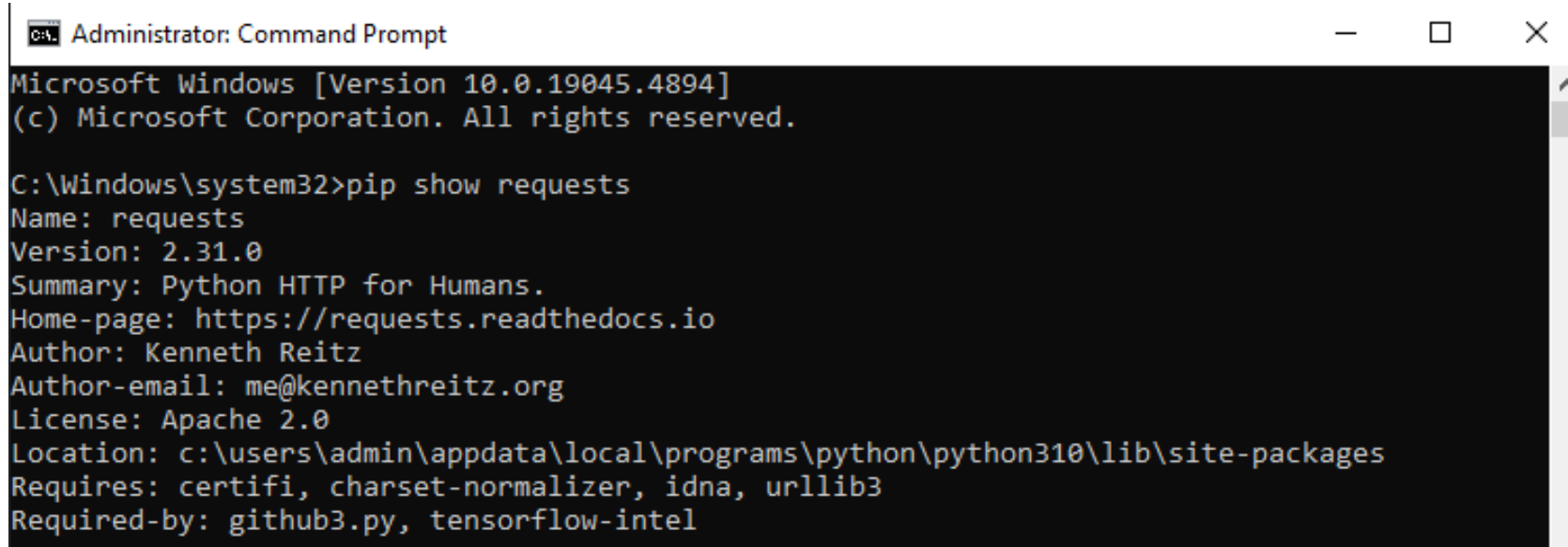
Kiểm tra thư viện 'request' đã được cài đặt trong Python

C1. Mở Python interactive shell (IDLE, terminal, hoặc command prompt) và thử nhập lệnh `import requests`. Nếu không có lỗi xảy ra, nghĩa là thư viện `requests` đã được cài đặt. Nếu thư viện chưa được cài, Python sẽ báo lỗi **ModuleNotFoundError**.

C2: Kiểm tra qua pip

Sử dụng **pip** (Python package installer) để kiểm tra xem thư viện đã được cài đặt hay chưa. Chạy lệnh dưới đây trong terminal hoặc command prompt:

`pip show requests`

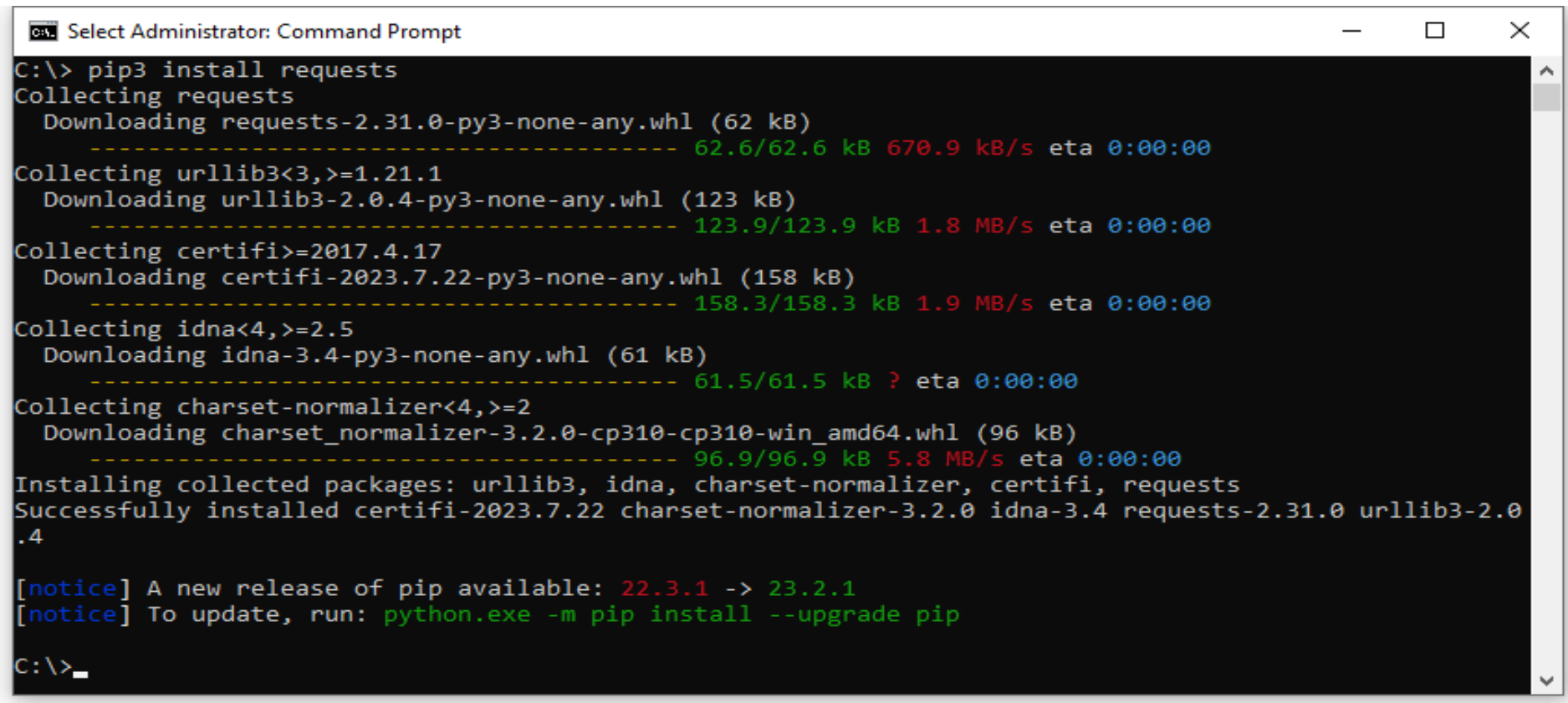


```
C:\Windows\system32>pip show requests
Name: requests
Version: 2.31.0
Summary: Python HTTP for Humans.
Home-page: https://requests.readthedocs.io
Author: Kenneth Reitz
Author-email: me@kennethreitz.org
License: Apache 2.0
Location: c:\users\admin\appdata\local\programs\python\python310\lib\site-packages
Requires: certifi, charset-normalizer, idna, urllib3
Required-by: github3.py, tensorflow-intel
```

Cài đặt gói request.

Cài đặt thư viện 'request' trong Python, hãy mở cửa sổ dòng lệnh hoặc CMD ở chế độ Administrator và gõ lệnh:

```
pip3 install requests
```



```
C:\> pip3 install requests
Collecting requests
  Downloading requests-2.31.0-py3-none-any.whl (62 kB)
----- 62.6/62.6 kB 670.9 kB/s eta 0:00:00
Collecting urllib3<3,>=1.21.1
  Downloading urllib3-2.0.4-py3-none-any.whl (123 kB)
----- 123.9/123.9 kB 1.8 MB/s eta 0:00:00
Collecting certifi>=2017.4.17
  Downloading certifi-2023.7.22-py3-none-any.whl (158 kB)
----- 158.3/158.3 kB 1.9 MB/s eta 0:00:00
Collecting idna<4,>=2.5
  Downloading idna-3.4-py3-none-any.whl (61 kB)
----- 61.5/61.5 kB ? eta 0:00:00
Collecting charset-normalizer<4,>=2
  Downloading charset_normalizer-3.2.0-cp310-cp310-win_amd64.whl (96 kB)
----- 96.9/96.9 kB 5.8 MB/s eta 0:00:00
Installing collected packages: urllib3, idna, charset-normalizer, certifi, requests
Successfully installed certifi-2023.7.22 charset-normalizer-3.2.0 idna-3.4 requests-2.31.0 urllib3-2.0.4

[notice] A new release of pip available: 22.3.1 -> 23.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\>_
```

Hình 2.4. Màn hình cài đặt gói request.

Ví dụ 2.2.5.1. Code chương trình:

```
1  import requests
2
3  # Thực hiện yêu cầu HTTP đến URL 'https://jsonplaceholder.typicode.com/posts'
4  response = requests.get('https://jsonplaceholder.typicode.com/posts')
5
6  # Kiểm tra xem yêu cầu thành công hay không
7  if response.status_code == 200:
8      # Nếu yêu cầu thành công, lấy dữ liệu JSON từ phản hồi
9      json_data = response.json()
10
11     # In ra màn hình thông tin các bài đăng từ dữ liệu JSON
12     for post in json_data:
13         print("ID bài đăng:", post['id'])
14         print("Tiêu đề:", post['title'])
15         print("Nội dung:", post['body'])
16         print("=====")
17 else:
18     print("Yêu cầu không thành công. Mã trạng thái:", response.status_code)
```

Kết quả thực hiện chương trình:

ID bài đăng: 1 Tiêu đề: sunt aut facere repellat provident
occaecati excepturi optio reprehenderit Nội dung: quia et
suscipit suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam nostrum rerum est autem
sunt rem eveniet architecto =====

ID bài đăng: 2 Tiêu đề: qui est esse Nội dung: est rerum
tempore vitae sequi sint nihil reprehenderit dolor beatae ea
dolores neque fugiat blanditiis voluptate porro vel nihil
molestiae ut reiciendis qui aperiam non debitis possimus qui
neque nisi nulla =====

• • •

=====

ID bài đăng: 4 Tiêu đề: eum et est occaecati Nội dung: ullam et
saepe reiciendis voluptatem adipisci sit amet autem assumenda
provident rerum culpa...

Ví dụ 2.2.5.2.

Có URL của World Bank về dữ liệu dân số 2020 tại:

<http://api.worldbank.org/v2/country/all/indicator/SP.POP.TOTL?date=2020&format=xml>

* Tải và phân tích dữ liệu dân số từ URL trên

Điểm giống khác nhau giữa JSON và XML



JSON (JavaScript Object Notation) và XML (Extensible Markup Language) đều là các định dạng dữ liệu được sử dụng để lưu trữ và truyền thông tin giữa các ứng dụng và hệ thống. Tuy nhiên, chúng có những điểm khác nhau sau đây:

Giống nhau

Đặc tính	XML	JSON
Trao đổi dữ liệu	Được thiết kế để trao đổi dữ liệu giữa các hệ thống	Được thiết kế để trao đổi dữ liệu giữa các hệ thống
Định dạng văn bản	Cả XML và JSON đều là định dạng văn bản, có thể đọc được bằng văn bản thuần (text-based)	Định dạng văn bản, có thể đọc được bằng văn bản thuần (text-based)
Hỗ trợ phân cấp	Hỗ trợ cấu trúc dữ liệu phân cấp, các phần tử có thể lồng nhau	Hỗ trợ cấu trúc dữ liệu phân cấp, các đối tượng có thể lồng nhau
Khả năng truyền tải qua mạng	Được sử dụng để truyền tải dữ liệu qua mạng giữa máy chủ và máy khách	Được sử dụng để truyền tải dữ liệu qua mạng giữa máy chủ và máy khách
Độc lập nền tảng	Độc lập với nền tảng và ngôn ngữ lập trình	Độc lập với nền tảng và ngôn ngữ lập trình

Khác biệt giữa JSON và XML



Đặc tính	XML	JSON
Định dạng	Định dạng dựa trên thẻ (tag-based)	Định dạng dựa trên cặp khóa-giá trị (key-value)
Mục đích ban đầu	Được thiết kế để chứa dữ liệu có cấu trúc phức tạp và siêu dữ liệu	Được thiết kế cho dữ liệu nhẹ trong ứng dụng web
Dễ đọc	Khá khó đọc do có nhiều thẻ đóng mở	Dễ đọc hơn do cú pháp đơn giản với {} và []
Độ dài dữ liệu	Dữ liệu thường dài hơn vì có thẻ mở và thẻ đóng	Ngắn gọn hơn vì không có thẻ đóng mở
Hỗ trợ kiểu dữ liệu	Không hỗ trợ trực tiếp, nhưng có thể xác định qua XML Schema (XSD)	Hỗ trợ các kiểu như chuỗi, số, mảng, boolean, null
Hỗ trợ thuộc tính	Có hỗ trợ thuộc tính trong các thẻ	Không hỗ trợ thuộc tính trực tiếp, dùng đối tượng lồng
Khả năng xác thực	Có thể xác thực với DTD hoặc XML Schema	Không có xác thực tích hợp, cần JSON Schema nếu muốn
Namespace	Hỗ trợ namespace để tránh xung đột thẻ	Không có namespace
Ứng dụng trong API	Thường được dùng trong các API cũ (SOAP)	Phổ biến trong các API hiện đại (REST)
Công cụ phân tích	Công cụ phân tích phức tạp, xử lý chậm hơn	Công cụ phân tích nhanh hơn, dễ thao tác với JavaScript
Ứng dụng phổ biến	Lưu trữ dữ liệu cấu trúc phức tạp, truyền thông trong hệ thống lớn	Truyền dữ liệu nhẹ giữa máy khách và máy chủ trong web

XML Schema là một sự phát triển sau này của XML để khắc phục các hạn chế của DTD(Document Type Definition)

<https://www.w3.org/TR/xmlschema-1/>

Khi nào nên sử dụng JSON?

- **Truyền dữ liệu giữa các ứng dụng web:** JSON là một định dạng dữ liệu rất phổ biến trong các ứng dụng web và thường được sử dụng để truyền dữ liệu giữa các ứng dụng web.
- **Lưu trữ cấu trúc dữ liệu đơn giản:** JSON thường được sử dụng để lưu trữ cấu trúc dữ liệu đơn giản như các danh sách, bảng thông tin, đối tượng hoặc mảng.
- **Sử dụng với các ngôn ngữ lập trình web:** JSON được hỗ trợ bởi hầu hết các ngôn ngữ lập trình web như JavaScript, Python, Ruby, PHP, v.v.
- **Dữ liệu nhẹ:** JSON có dung lượng nhẹ hơn so với các định dạng dữ liệu khác như XML, do đó thường được sử dụng để truyền dữ liệu giữa các ứng dụng web.

⇒ *Nên sử dụng JSON khi cần truyền dữ liệu giữa các ứng dụng web hoặc lưu trữ cấu trúc dữ liệu đơn giản, đặc biệt là khi sử dụng các ngôn ngữ lập trình web và cần một định dạng dữ liệu nhẹ.*

Câu hỏi thảo luận

1. Anh/Chị hãy trình bày khái niệm XML là gì?
2. Khái niệm DOM?
3. Cú pháp tài liệu XML?
4. Khái niệm JSON
5. Hạn chế của việc triển khai JSON trong Python?
6. API XML là gì?
7. API JSON là gì?
8. Để phân tích tập tin XML có thể sử dụng các thư viện nào trong Python?

Bài tập vận dụng

Làm các bài tập chương 2: 2.7, đến 2.13 trong TLHT /trên LMS